Tracking and Grasping AprilTags

Tracking and Grasping AprilTags

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

1. Course Content

This section's feature acquires images through the camera and recognizes the AprilTags in the images. When the AprilTag is moved by hand, the robotic arm follows along, keeping the center point of the AprilTag in the middle of the image; when the robotic arm stops tracking, the robotic arm goes to grasp the AprilTag wooden block and then places it in a set position.

The wooden block used in this section: **40x40x40mm AprilTag wooden block**.

2. Preparation

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=apriltag_follow_grasp
```

3. Case Demonstration

- After the program runs, hold the **40x40x40mm AprilTag wooden block** provided with the product so that it appears in the image, as shown in the figure below



- **Hand-held guidance**: hold the wooden block with the AprilTag and slowly pan or move it up and down in front of the robotic arm camera's field of view;
- **Observation state indicator box**: when the AprilTag is **not aligned**, the box and crosshair are blue; when the AprilTag is **aligned**, the box switches to green;
- The chassis automatically advances smoothly toward the target wooden block;
- After reaching the grasping distance, the chassis stops steadily, and the robotic arm adaptively aligns with the wooden block's deflection angle and grasps it;

> **Note**
>
> - In rare cases, if the target AprilTag is at a position where the robotic arm end-effector cannot obtain an inverse kinematics solution, abnormal grasping by the robotic arm may occur. You can adjust the height at which the AprilTag is located to solve this problem.

4. Core Source Code Analysis

- Program path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/apriltag_follow_grasp.py
```

_grasp_worker function (grasping worker thread):

This function is the main control thread of the grasping sequence, sequentially completing obtaining the AprilTag pose, angle calculation, inverse kinematics solution, robotic arm motion, gripper closing, placement, and other steps.

- `time.sleep(0.5)`: waits for the camera image to stabilize before starting to grasp
- `self._latest_tags`: traverses the latest detected AprilTag list and selects the first target whose pose is successfully solved
- `_estimate_tag_pose(t)`: calls solvePnP to estimate the pose of the AprilTag in the camera coordinate frame, returning the translation vector `tvec` and rotation vector `rvec`
- `get_apriltag_pose_in_measure(cam_x, cam_y, cam_z)`: converts the position in the camera coordinate frame to the measure coordinate frame
- `cv2.Rodrigues(rvec)`: converts the rotation vector to the rotation matrix R
- `math.atan2(R[1, 0], R[0, 0])`: extracts the in-plane rotation angle (yaw) around the camera optical axis from the rotation matrix
- `normalize_angle_to_0_45(rvec_yaw_deg)`: normalizes the yaw angle to the range 0°~45°, used to determine the gripper rotation direction
- `grasp_offset`: grasping offset `[gx, gy, gz, pitch]`, compensating for the difference between the camera-detected position and the actual grasping position
- `arm_kinematics.inverse_kinematics(...)`: calls the inverse kinematics solver, taking the target position and pitch angle as input, and outputting 5 joint angles
- `result.approximate` / `result.adjusted_pitch`: handles the cases of approximate solutions and automatic pitch angle adjustment
- `joints[0:3] = round(ik_joints[0:3])`: rounds the J1~J4 joint angles obtained from the inverse solution to integers
- `joints[4] = round(90 + angle_normalized)`: the J5 joint angle is calculated from the normalized yaw angle, so that the gripper rotation aligns with the AprilTag direction
- `joints[5] = 0`: the J6 gripper angle is set to 0° (open state)
- `pubSix_Arm(joints, runtime=1500)`: publishes the 6-joint control command, and moves to the grasping pose
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=gripping_angle, time=1000))`: controls the gripper to close individually
- According to `tag_id` (1~4), selects a different placement joint angle `down_joint`, and places the AprilTag at the corresponding position
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))`: opens the gripper to release the object
- `pubSix_Arm(observation_pose)`: returns the robotic arm to the observation pose
- `finally: self._reset_grasp_state()`: regardless of success or failure, resets the grasping state at the end

_arm_track_center function (robotic arm tracking and centering):

This function is called continuously during the tracking phase, keeping the AprilTag center point at the image center by adjusting the robotic arm joint angles.

- `self._ema_cx / self._ema_cy`: the AprilTag center pixel coordinates after EMA (exponential moving average) filtering, smoothing out jitter
- `img_cx = 320 / img_cy = 240`: image center coordinates (640×480 resolution)
- `track_pixel_tolerance`: pixel alignment tolerance; no adjustment is made when the error is within this range
- `gaze_step`: the joint angle step (degrees) for each adjustment
- `error_x = self._ema_cx - img_cx`: calculates the horizontal pixel error
- `error_y = self._ema_cy - img_cy`: calculates the vertical pixel error
- `self._is_stable_aligned`: if already stably aligned, returns directly without further adjustment
- Joint state initialization: preferentially uses the joint angles subscribed in real time, and falls back to the observation pose when there is no feedback
- **X axis prioritized**: when the horizontal error exceeds the tolerance, adjusts J1 (base rotation joint); when `error_x > 0`, increases the angle, otherwise decreases the angle, limited to the range 0~180°
- **Y axis adjustment**: after horizontal alignment, when the vertical error exceeds the tolerance, adjusts J4 (pitch joint); when `error_y > 0`, decreases the angle, otherwise increases the angle
- **J4 limit handling**: when the J4 angle is below -20°, adjusts J2 (big arm joint) instead for compensation, preventing J4 from exceeding the mechanical limit
- `SingleJoint_pub.publish(ArmJoint(id=..., joint=..., time=500))`: publishes a single-joint control command, adjusting only one joint dimension at a time

```
    def _grasp_worker(self):
        """Worker thread: execute the grasp sequence (IK -> move to target -> close gripper
-> place -> return to observation pose)"""
        try:
            time.sleep(0.5)  # wait for the camera to stabilize

            # Get the latest tag data
            tag = None
            pose = None
            for t in self._latest_tags:
                tvec, rvec = self._estimate_tag_pose(t)
                if tvec is None:
                    continue
                cam_x, cam_y, cam_z = tvec[0][0], tvec[1][0], tvec[2][0]
                pose = self.get_apriltag_pose_in_measure(cam_x, cam_y, cam_z)
                if pose is not None:
                    tag = t
                    # Extract the in-plane rotation angle around the camera optical axis
(yaw) from rvec
                    R, _ = cv2.Rodrigues(rvec)
                    rvec_yaw_deg = math.degrees(math.atan2(R[1, 0], R[0, 0]))
                    break

            if tag is None or pose is None:
                self.get_logger().error("Tag lost before grasp")
                return
```

```
            # Extract the in-plane rotation angle around the camera optical axis from the
solvePnP rvec
            angle_normalized = normalize_angle_to_0_45(rvec_yaw_deg)
            self.get_logger().info(f"Tag rvec yaw: {rvec_yaw_deg:.1f}° -> normalized:
{angle_normalized:.1f}°")

            # Apply grasp_offset to the IK target position
            mx, my, mz = pose
            gx, gy, gz = self.grasp_offset[0], self.grasp_offset[1], self.grasp_offset[2]
            self.get_logger().info(
                f"measure pose: x={mx:.3f}, y={my:.3f}, z={mz:.3f}; "
                f"IK target: x={mx+gx:.3f}, y={my+gy:.3f}, z={mz+gz:.3f}, pitch=
{self.grasp_offset[3]:.1f}°"
            )

            result: KinResult = self.arm_kinematics.inverse_kinematics(
                target_x=mx + gx,
                target_y=my + gy,
                target_z=mz + gz,
                target_pitch=self.grasp_offset[3],
                seed_angles_deg=None,
            )

            if not result.success:
                self.get_logger().error(f"Grasp IK failed: {result.message}")
                return

            if result.approximate:
                self.get_logger().warn(f"IK approximate solution, error=
{result.position_error:.4f}m")
            if result.adjusted_pitch is not None:
                self.get_logger().warn(f"IK pitch sweep, adjusted pitch=
{result.adjusted_pitch:.1f}°")

            # Build the joint angles
            ik_joints = result.data
            joints = [0, 0, 0, 0, 0, 0]
            joints[0] = round(ik_joints[0])
            joints[1] = round(ik_joints[1])
            joints[2] = round(ik_joints[2])
            joints[3] = round(ik_joints[3])
            joints[4] = round(90+angle_normalized)
            joints[5] = 0

            self.get_logger().info(f"Grasp joints: {joints}")
            self.pubSix_Arm(joints,runtime=1500)
            time.sleep(3.0)
            self.SingleJoint_pub.publish(ArmJoint(id=6, joint=self.gripping_angle,
time=1000))
            time.sleep(3.0)

            # Move to the release position
```

```
            if tag.tag_id == 1:
                down_joint = [180, 35, 70, 5, 60, self.gripping_angle]
            elif tag.tag_id == 2:
                down_joint = [180, 35, 60, 0, 90, self.gripping_angle]
            elif tag.tag_id == 3:
                down_joint = [0, 30, 70, 2, 90, self.gripping_angle]
            elif tag.tag_id == 4:
                down_joint = [0, 43, 48, 6, 90, self.gripping_angle]
            else:
                down_joint = [180, 35, 70, 5, 60, self.gripping_angle]
            self.pubSix_Arm(down_joint)
            time.sleep(2.5)

            self.SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))
            time.sleep(2.0)

            self.pubSix_Arm(list(self.observation_pose))
            time.sleep(2.5)

            self.get_logger().info("Grasp sequence complete")
        finally:
            self._reset_grasp_state()
def _arm_track_center(self):
        if self._ema_cx is None or self._ema_cy is None:
            return

        img_cx = 320
        img_cy = 240
        tolerance =
self.get_parameter("track_pixel_tolerance").get_parameter_value().integer_value
        step = self.get_parameter("gaze_step").get_parameter_value().integer_value
        move_time = 500  # single-joint move time (ms)

        error_x = self._ema_cx - img_cx
        error_y = self._ema_cy - img_cy

        # Do not adjust while stably aligned
        if self._is_stable_aligned:
            return

        # Initialize/sync the current joint state: prefer the real-time subscribed joint
angles; fall back to observation_pose when no feedback is available
        if self._joint_state_received and self._current_joint_states is not None:
            self._track_joints = list(self._current_joint_states)
        elif self._track_joints is None:
            self._track_joints = list(self.observation_pose)

        # Prioritize the X axis (joint1 base rotation); adjust only one dimension at a time
        if abs(error_x) > tolerance:
            if error_x > 0:
                self._track_joints[0] += step
            else:
                self._track_joints[0] -= step
```

```
            self._track_joints[0] = max(0, min(180, self._track_joints[0]))
            self.SingleJoint_pub.publish(ArmJoint(id=1, joint=int(self._track_joints[0]),
time=move_time))
            return

        # X axis aligned; now adjust the Y axis (joint4, or joint2 when the limit is
reached)
        if abs(error_y) > tolerance:
            if error_y > 0:
                self._track_joints[3] -= step
            else:
                self._track_joints[3] += step
            # When joint4 reaches its lower limit, adjust joint2 instead
            if self._track_joints[3] < -20:
                self._track_joints[3] = -20
                self._track_joints[1] -= step
                self._track_joints[1] = max(0, min(180, self._track_joints[1]))
                self.SingleJoint_pub.publish(ArmJoint(id=2,
joint=int(self._track_joints[1]), time=move_time))
            else:
                self._track_joints[3] = max(-5, min(90, self._track_joints[3]))
                self.SingleJoint_pub.publish(ArmJoint(id=4,
joint=int(self._track_joints[3]), time=move_time))
```

- Parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

Parameter Description:

- `tag_size: 0.025`: the physical side length of the Aruco marker (meters), used for solvePnP pose estimation
- `observation_pose: [90, 150, 12, 20, 90, 30]`: the 6 joint angles (degrees) of the observation pose, at which the robotic arm has the best field of view
- `grasp_offset: [0.0, 0.0, 0.0, 0.0]`: grasping offset `[x, y, z, pitch]`, compensating for the difference between the camera-detected position and the actual grasping position; this parameter can be adjusted when there is a grasping deviation
- `gripping_angle: 130`: the angle (degrees) when the gripper is closed; the larger the value, the stronger the gripping force
- `track_distance_threshold: 0.16`: tracking distance threshold (meters); when the AprilTag distance exceeds this value, the chassis automatically advances toward the target
- `track_pixel_tolerance: 20`: pixel alignment tolerance; alignment is considered achieved when the deviation between the AprilTag center and the image center is less than this value
- `gaze_step: 3`: the step (degrees) of each joint adjustment; the larger the value, the faster the tracking response but the more likely overshoot
- `ema_alpha: 0.3`: EMA filter coefficient (0~1); the smaller the value, the smoother the filtering but the

slower the response

- `alignment_stable_count: 3`: consecutive alignment count threshold; tracking is considered stable only after reaching this count, which can then trigger grasping

```
apriltag_follow_grasp:
  ros__parameters:
    tag_size: 0.025                          # AprilTag side length (meters)
    observation_pose: [90, 150, 12, 20, 90, 30]
    grasp_offset: [0.0, 0.0, 0.0, 0.0]    # x,y,z axis offset distance and included angle
    gripping_angle: 130                      # gripper grasping angle, in degrees, integer
    track_distance_threshold: 0.16           # tracking distance threshold (meters); the
chassis moves when exceeded
    track_pixel_tolerance: 20                # pixel alignment tolerance
    gaze_step: 3                              # joint adjustment step per cycle (degrees)
    ema_alpha: 0.3                            # EMA filter coefficient (0~1), the smaller the
smoother
    alignment_stable_count: 3                 # consecutive alignment count threshold;
considered stable only after reaching it
```